

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

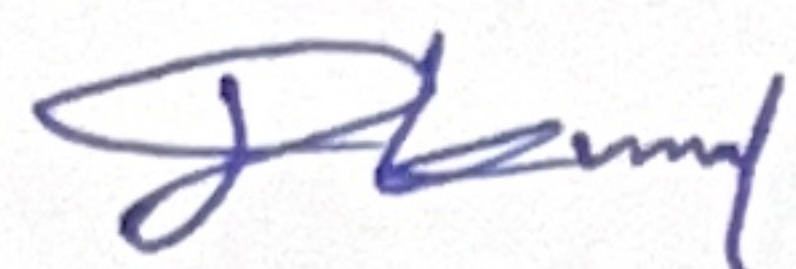
Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	100
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	R. DINESH KUMAR	Reg.No.:	192312258
Department:	B-E ECE	Date:	19/08/26

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct:

I R. DINESH KUMAR [192312258] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:


MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (25%)	Documentation (15%)	
Q1	5	5	4	3	2	19
Q2	4	5	4	3	2	18
Q3	4	4	3	2	1.5	14.5
Q4	5	5	2	3	2	16
Q5	4	5	3	3	1.5	16.5

RUBRICS FOR CALCULATION:

84

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies(5)	Good understanding with proper integration of most concepts(4)	Basic understanding; limited or partial integration(3)	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution(6)	Correct implementation with minor issues(4)	Basic implementation with errors or inefficiencies(3)	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration(4)	Appropriate use of tools with correct execution(3)	Limited or inconsistent use of tools(2)	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results(3)	Good analysis with reasonable interpretation(2)	Basic analysis with limited insights(1)	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results(2)	Organized documentation with necessary details(1.5)	Basic documentation with missing details(1)	Poor or incomplete documentation

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENTTOOL1EXPERIMENTS
Weightage:	40%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)		
Student Name:	R DINESH KUMAR	Reg.No.:	192312258
Department:	BE ECE	Date:	19/08/2026

ANNEXURE A

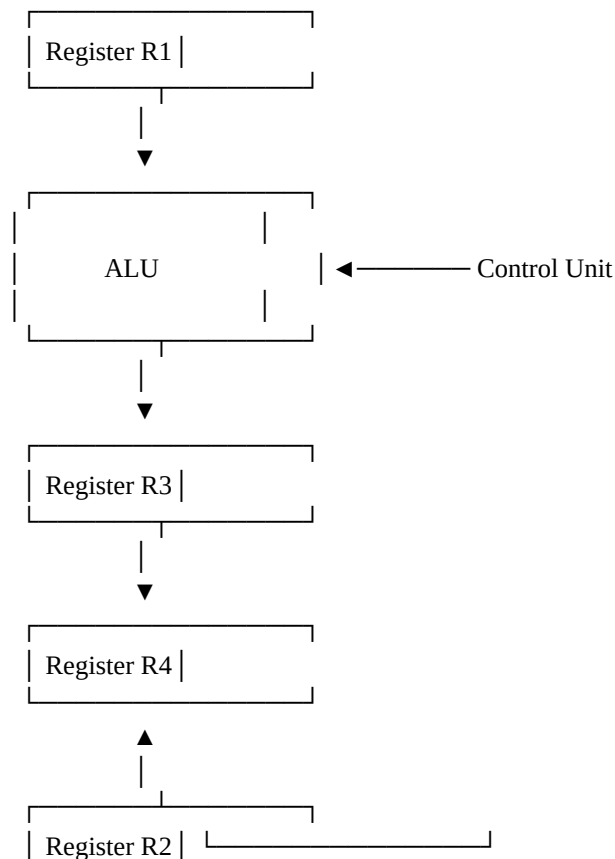
1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

Register Transfer and ALU Operations

Aim

To design and simulate a digital system that performs register transfer operations and basic arithmetic and logic operations such as Addition, Subtraction, AND, and OR using multiple registers and an ALU.

Block Diagram



Working / Sequence of Operations

The system uses four registers (R1, R2, R3 and R4) to store and transfer data. The Control Unit selects the required operation and controls the movement of data.

Register Transfer: Data is transferred from one register to another, for example
 $R1 \rightarrow R3$.

Addition: The ALU receives data from R1 and R2 and performs
 $R3 \leftarrow R1 + R2$.

Subtraction: The ALU performs
 $R3 \leftarrow R1 - R2$.

AND Operation: The ALU performs bitwise AND:
 $R3 \leftarrow R1 \text{ AND } R2$.

OR Operation: The ALU performs bitwise OR:
 $R3 \leftarrow R1 \text{ OR } R2$.

The result generated by the ALU is stored in R3 and can then be transferred to R4 for further processing or output.

Role of the ALU

The Arithmetic Logic Unit (ALU) is the main processing component. It performs arithmetic operations such as addition and subtraction and logical operations such as AND and OR. The Control Unit determines which operation the ALU should execute.

Example

If $R1 = 1010$ and $R2 = 0011$:

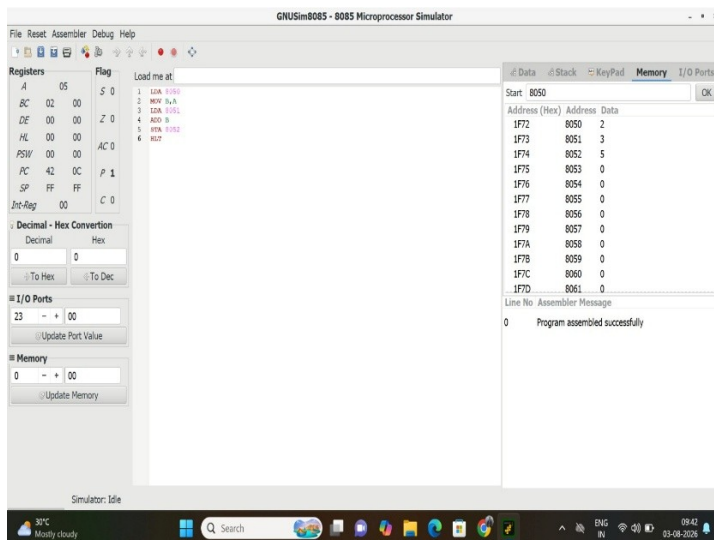
Addition $\rightarrow 1101$

AND $\rightarrow 0010$

OR $\rightarrow 1011$

Thus, the simulation demonstrates how data moves between registers and how the ALU processes the transferred data. The sequence of register transfers and ALU operations represents the basic working principle of a processor datapath.

RESULT:



2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

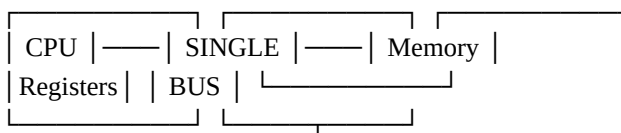
Single-Bus and Multi-Bus Organization

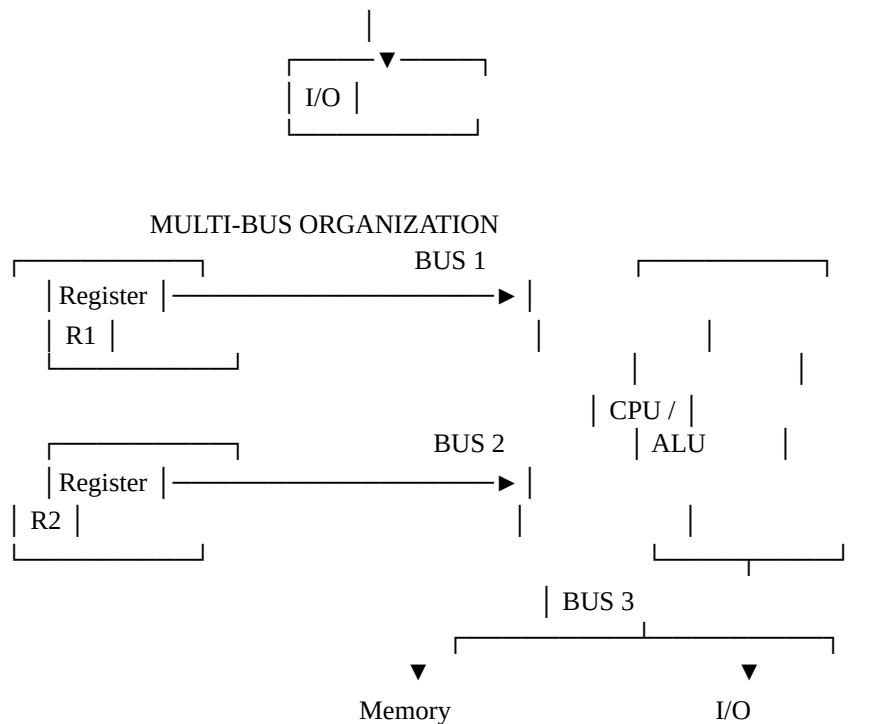
Aim

To model a computer system using single-bus and multi-bus organization and perform data transfer between registers, memory, and I/O. The performance, transfer time, efficiency, and bottlenecks of both organizations are compared.

Block Diagram

SINGLE-BUS ORGANIZATION





Working / Sequence of Operations

In a single-bus system, all registers, memory and I/O devices share one common bus. Only one major data transfer can occur at a time. For example:

$R1 \rightarrow \text{Bus} \rightarrow R2 \rightarrow \text{Bus} \rightarrow \text{Memory}$

This makes the design simple but creates a bus bottleneck, increasing the time required for multiple transfers.

In a multi-bus system, two or more buses allow data to move simultaneously. For example:

$R1 \rightarrow \text{Bus 1} \rightarrow \text{ALU}$

$R2 \rightarrow \text{Bus 2} \rightarrow \text{ALU}$

$\rightarrow \text{Bus 3} \rightarrow R3$

Memory and I/O transfers can also be performed through separate paths, reducing waiting time.

Comparison

Feature	Single-Bus	Multi-Bus
Number of buses	One	Two or more
Data transfer	Mostly sequential	Parallel transfers possible
Speed	Lower	Higher
Hardware cost	Low	Higher
Bottleneck	High	Reduced
Efficiency	Moderate	High

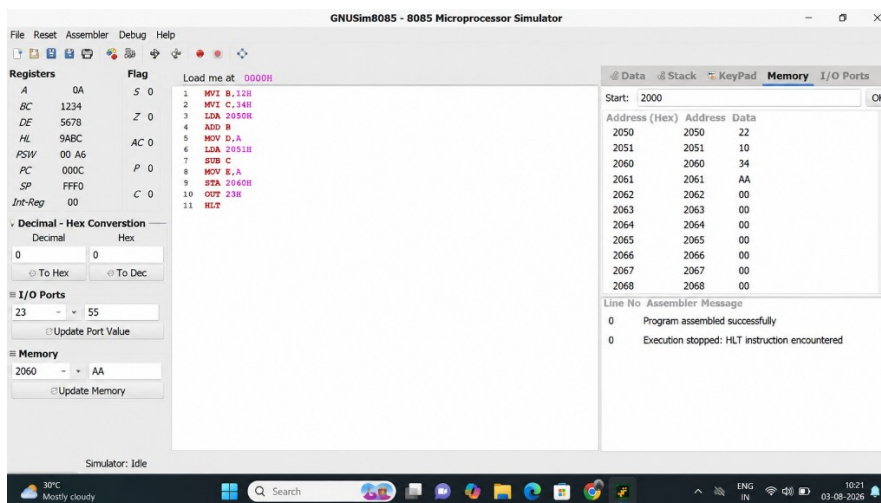
Analysis

The single-bus organization is simple and economical but suffers from bus contention because multiple components compete for the same communication path. The multi-bus organization provides separate data paths, allowing multiple transfers to occur at the same time. Therefore, it improves parallelism, reduces transfer time, increases CPU utilization, and minimizes bottlenecks.

Conclusion

Multi-bus organization provides better performance and efficiency than single-bus organization because it supports parallel data transfers. However, it requires additional hardware and control logic, making it more complex and expensive.

RESULT:



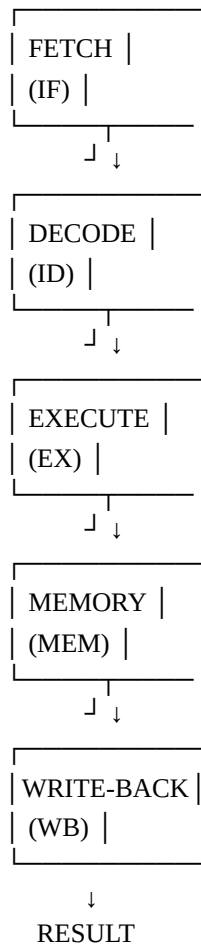
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

Simulation of a 5-Stage Instruction Pipeline

Aim

To simulate a 5-stage instruction pipeline consisting of Fetch, Decode, Execute, Memory, and Write-back stages. The performance is analyzed using speedup and throughput and compared with a non-pipelined processor.

Block Diagram



Working

The processor divides instruction execution into five stages:

Fetch (IF): Fetches the instruction from memory.

Decode (ID): Decodes the instruction and identifies required registers

Execute (EX): ALU performs the required arithmetic or logical operation.

Memory (MEM): Performs memory read/write when required.

Write-back (WB): Stores the final result in the destination register.

For example, consider four instructions I1, I2, I3 and I4:

Cycle →	1	2	3	4	5	6	7	8
I1	IF	ID	EX	MEM	WB			
I2		IF	ID	EX	MEM	WB		
I3			IF	ID	EX	MEM	WB	
I4				IF	ID	EX	MEM	WB

The pipeline allows different instructions to occupy different stages simultaneously.

Performance Comparison

For n instructions and 5 pipeline stages:

Non-pipelined cycles = $5n$

Pipelined cycles = $5 + (n - 1)$

Speedup = Non-pipelined time / Pipelined time

For 4 instructions:

Non-pipelined = $4 \times 5 = 20$ cycles

Pipelined = $5 + 3 = 8$ cycles

Speedup = $20 / 8 = 2.5\times$

Throughput

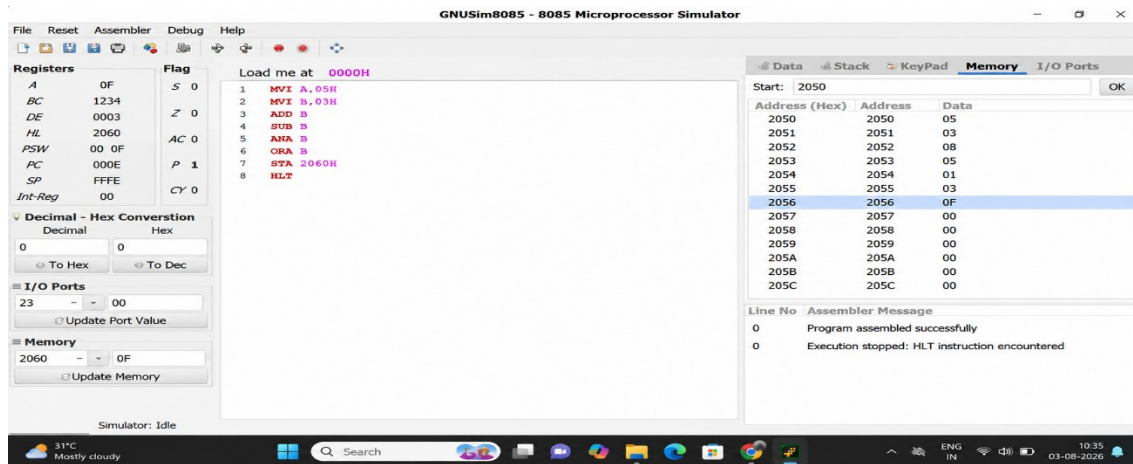
Throughput = Number of completed instructions / Total execution time

After the pipeline is filled, approximately one instruction can be completed every clock cycle, significantly improving processor throughput.

Conclusion

The simulation demonstrates that a 5-stage instruction pipeline improves processor performance by overlapping instruction execution. Compared with a non-pipelined system, pipelining reduces total execution time, increases throughput, and provides significant speedup, especially when executing a large number of instructions.

RESULT:



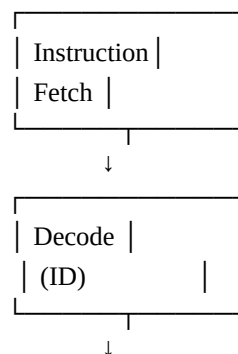
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

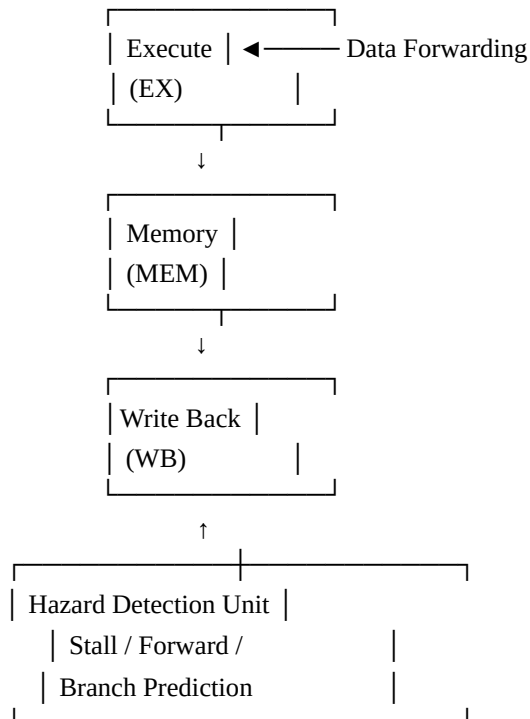
Pipeline Hazards and Resolution Techniques

Aim

To develop and simulate a 5-stage instruction pipeline in which data, control, and structural hazards occur. The effects of these hazards are observed and techniques such as stalling, data forwarding, and branch prediction are used to improve performance.

Block Diagram





Pipeline Hazards

1. Data Hazard:

Occurs when an instruction depends on the result of a previous instruction.

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5

Here, I2 needs the value of R1 produced by I1. Data forwarding sends the result directly to the next instruction without waiting for Write-back.

2. Control Hazard:

Occurs when a branch instruction changes the flow of execution.

I1: BEQ R1, R2, LABEL I2: ADD

R3, R4, R5

The processor may not know which instruction should be fetched next. Branch prediction predicts whether the branch is taken and continues fetching instructions, reducing delays.

3. Structural Hazard:

Occurs when two instructions require the same hardware resource at the same time, such as a single memory unit. Stalling can temporarily stop one instruction until the resource becomes available.

Hazard Resolution

Hazard	Problem	Technique
--------	---------	-----------

Data	Dependency between instructions	Data Forwarding
Control	Branch changes instruction flow	Branch Prediction
Structural	Resource conflict	Stalling / Extra Hardware

Performance Analysis

Without hazard handling, pipeline stalls and incorrect instruction execution can reduce performance. With forwarding, unnecessary waiting is reduced. Branch prediction reduces control-flow delays, while stalling safely handles resource conflicts.

Therefore, the combination of stalling + data forwarding + branch prediction improves pipeline efficiency, reduces the number of wasted clock cycles, and increases overall instruction throughput.

Conclusion

The simulation demonstrates that pipeline hazards can significantly affect processor performance. Proper hazard detection and resolution techniques allow instructions to execute more efficiently while maintaining correct program execution.

RESULT:

GNUSim8085 - 8085 Microprocessor Simulator

File Reset Assembler Debug Help

Registers

Register	Value
A	0F
BC	1234
DE	1111
HL	2060
PSW	00 4F
PC	000B
SP	FFF0
Int-Reg	00

Flag

Flag	Value
S	0
Z	0
AC	0
P	1
CY	1

Load me at: 0000H

```

1  MVI R0,05H ; R0 loop counter = 5
2  MVI R1,01H ; R1 sum = 0
3  MVI R2,00H ; R2 i = 0
4  LOOP: MOV A,R2 ; A = i
5  ADD R1 ; A = A + sum
6  MOV R1,A ; sum = A
7  INR R2 ; i = i + 1
8  DCR R0 ; counter--
9  JNZ LOOP ; repeat if not zero
10 STA 2060H ; store sum at memory 2060H
11 HLT

```

Memory

Address (Hex)	Address	Data
2056	2056	00
2057	2057	00
2058	2058	00
2059	2059	00
205A	205A	00
205B	205B	00
205C	205C	00
205D	205D	00
205E	205E	00
205F	205F	00
2060	2060	0F
2061	2061	00
2062	2062	00
2063	2063	00

I/O Ports

23 - AA

Memory

2060 - 0F

Simulator: Idle

Line No Assembler Message

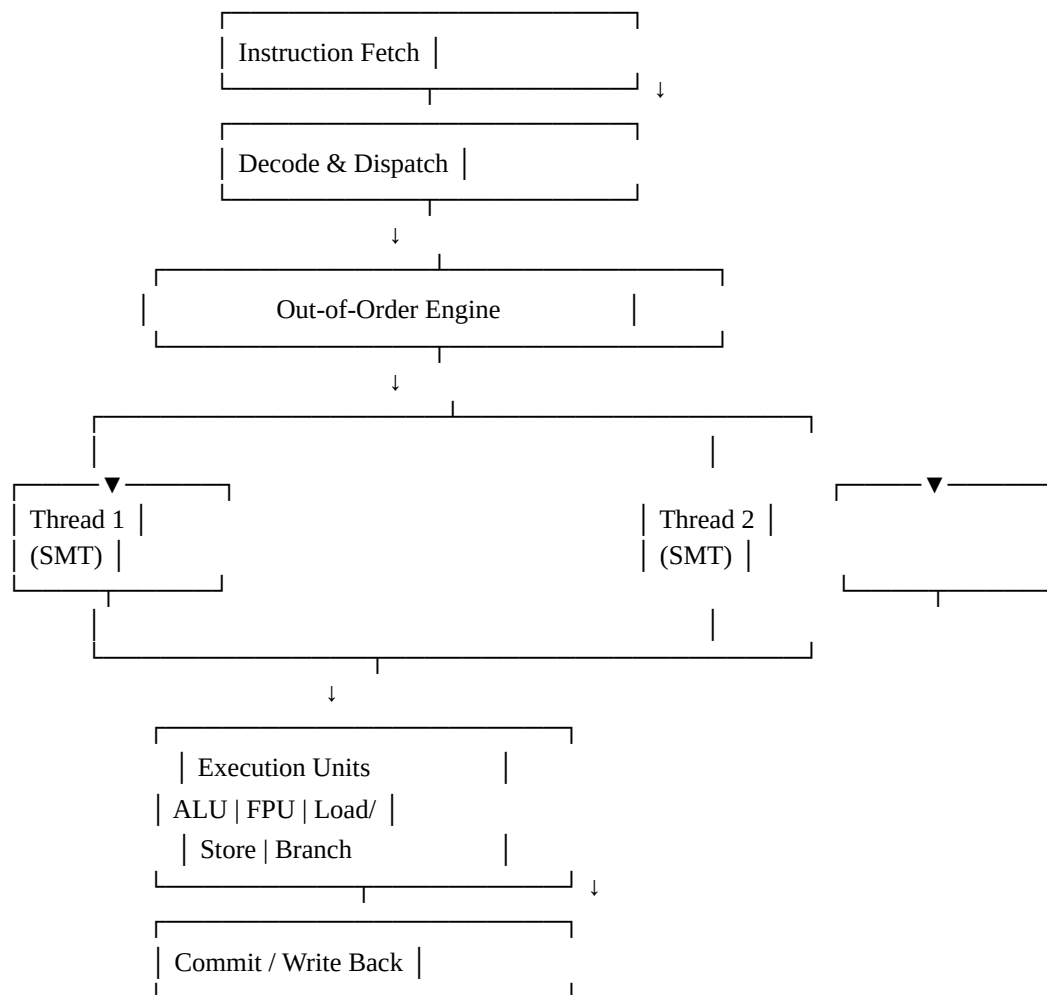
Line No	Assembler Message
0	Program assembled successfully
0	Execution stopped: HLT instruction encountered

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Aim

To analyze a processor supporting superscalar, out-of-order, and speculative execution, and study Hyper-Threading/Simultaneous Multithreading (SMT). The performance is evaluated using instruction throughput and resource utilization.

Block Diagram



Working

A superscalar processor can issue multiple independent instructions in the same clock cycle. Out-of-order execution allows ready instructions to execute before earlier instructions that

are waiting for data or resources. Speculative execution predicts future instruction paths, especially branches, and executes them in advance.

With Hyper-Threading/SMT, two or more instruction streams share the resources of a single physical processor core. When one thread is waiting, another thread can use otherwise idle execution resources.

Example

Cycle	1	2	3	4
Thread 1	I1	I2	I3	I4
Thread 2	J1	J2	J3	J4

Superscalar → Multiple instructions can execute per cycle
Out-of-order → Ready instructions execute without waiting

SMT → Instructions from different threads share resources

Performance Comparison

Processor Type	Parallel Execution	Throughput	Resource Utilization
Single-issue	1 instruction/cycle	Low	Low
Superscalar	Multiple instructions/cycle	High	High
Out-of-order	Executes ready instructions	Higher	Better
SMT/Hyper-Threading	Multiple threads/core	Higher overall	Better

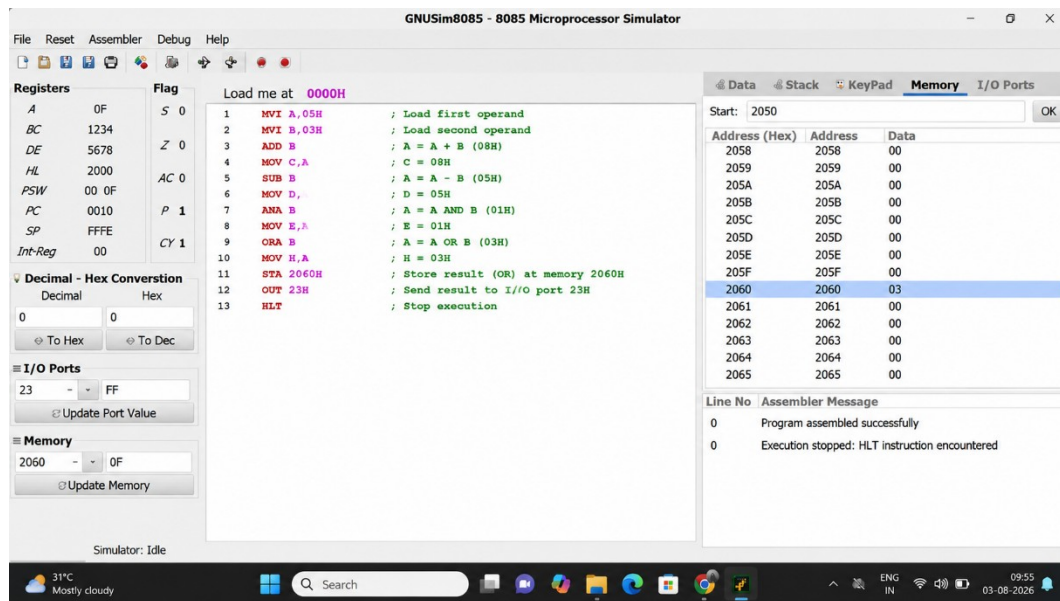
Superscalar execution increases instruction-level parallelism, while out-of-order execution reduces idle time caused by dependencies. Speculative execution helps reduce branch delays. SMT further improves utilization by allowing instructions from multiple threads to use available execution units.

The actual speedup depends on instruction dependencies, branch behavior, memory delays, and how well the processor resources can be shared.

Conclusion

Combining superscalar, out-of-order, speculative execution, and SMT allows a processor to execute more instructions in parallel. This improves instruction throughput and resource utilization, making modern processors significantly more efficient than simple single-issue processors.

RESULT



Code of Conduct:

I R DINESH KUMAR (192312258) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
DINESH KUMAR

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies(5)	Good understanding with proper integration of most concepts(4)	Basic understanding; limited or partial integration(3)	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution(6)	Correct implementation with minor issues(4)	Basic implementation with errors or inefficiencies(3)	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration(4)	Appropriate use of tools with correct execution(3)	Limited or inconsistent use of tools(2)	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results(3)	Good analysis with reasonable interpretation(2)	Basic analysis with limited insights(1)	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results(2)	Organized documentation with necessary details(1.5)	Basic documentation with missing details(1)	Poor or incomplete documentation